



Python String

Last Updated : 28 Mar, 2026

Strings are sequence of characters written inside quotes. It can include letters, numbers, symbols and spaces. Python does not have a separate character type.

- A single character is treated as a string of length one.
- Strings are commonly used for text handling and manipulation.

Creating a String

Strings can be created using either single ('...') or double ("...") quotes. Both behave the same.

Example: Creating two equivalent strings one with single and other with double quotes.

```
s1 = 'GfG'
s2 = "GfG"
print(s1)
print(s2)
```



Output

```
GfG
GfG
```

Multi-line Strings

Use triple quotes ('''...''') or ("""...""") for strings that span multiple lines. Newlines are preserved.

Example: Define and print multi-line strings using both styles.

```
s = """I am Learning
Python String on GeeksforGeeks"""
print(s)

s = '''I'm a
Geek'''
print(s)
```



Output

```
I am Learning
Python String on GeeksforGeeks
I'm a
Geek
```

Accessing characters in String

Strings are indexed sequences. Positive indices start at 0 from the left, negative indices start at -1 from the right as represented in below image:

0	1	2	3	4	5	6	7	8	9	10	11	12
G	E	E	K	S	F	O	R	G	E	E	K	S
-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Indices of string in reverse

Example 1: Access specific characters through positive indexing.

```
s = "GeeksforGeeks"  
print(s[0])  
print(s[4])
```

× ▶ 📄

Output

```
G  
S
```

Note: Accessing an index out of range will cause an `IndexError`. Only integers are allowed as indices and using a float or other types will result in a `TypeError`.

Example 2: Read characters from the end using [negative indices](#).

```
s = "GeeksforGeeks"  
print(s[-10])  
print(s[-5])
```

× ▶ 📄

Output

```
k  
G
```

String Slicing

[Slicing](#) is a way to extract a portion of a string by specifying the start and end indexes. The syntax for slicing is `string[start:end]`, where start is starting index and end is stopping index (excluded).

Example: this example demonstrates slicing through range and reversing a string.

```
s = "GeeksforGeeks"  
print(s[1:4])  
print(s[:3])  
print(s[3:])  
print(s[::-1])
```

× ▶ 📄

Output

```
EEK  
Gee
```

ksforGeeks
skeeGrofskeeG

String Iteration

Strings are iterable, one can loop through characters one by one.

Example: Here, it prints each character on its own line.

```
s = "Python"
for char in s:
    print(char)
```

× ▶ 📄

Output

P
y
t
h
o
n

Explanation: for loop pulls characters in order and each iteration prints the next character.

String Immutability

Strings are immutable, which means that they cannot be changed after they are created. If we need to manipulate strings then we can use methods like concatenation, slicing or formatting to create new strings based on original.

Example: In this example we are changing first character by building a new string.

```
s = "geeksforGeeks"
s = "G" + s[1:]
print(s)
```

× ▶ 📄

Output

GeeksforGeeks

Deleting a String

It's not possible to delete individual characters from a string since strings are immutable. However, we can delete an entire string variable using the [del](#) keyword.

Example: Here, we are using del keyword to delete a string.

```
s = "GfG"
del s
```

📄

Note: After deleting the string if we try to access *s* then it will result in a *NameError* because variable no longer exists.

Updating a String

As strings are immutable, “updates” create new strings using slicing or methods such as [replace\(\)](#).

Example: This code fixes the first letter and replace a word.

```
s = "hello geeks"
s1 = "H" + s[1:]
s2 = s.replace("geeks", "GeeksforGeeks")
print(s1)
print(s2)
```



Output

```
Hello geeks
hello GeeksforGeeks
```

Explanation:

- **s1**: slice from index 1 onward and prepend "H".
- **s2**: `replace("geeks", "GeeksforGeeks")` returns a new string.

Common String Methods

Python provides various built-in methods to manipulate strings. Below are some of the most useful methods:

1. `len()`: returns the total number of characters in a string (including spaces and punctuation).

```
s = "GeeksforGeeks"
print(len(s))
```



Output

```
13
```

2. `upper()` and `lower()`: `upper()` method converts all characters to uppercase whereas, `lower()` method converts all characters to lowercase.

```
s = "Hello World"
print(s.upper())
print(s.lower())
```



Output

```
HELLO WORLD
hello world
```

3. `strip()` and `replace()`: `strip()` removes leading and trailing whitespace from the string and `replace()` replaces all occurrences of a specified substring with another.

```
s = "  Gfg  "
print(s.strip())

s = "Python is fun"
print(s.replace("fun", "awesome"))
```



Output

Gfg
Python is awesome

To learn more about string methods, please refer to [Python String Methods](#).

Concatenating and Repeating Strings

We can concatenate strings using + operator and repeat them using * operator.

1. Strings can be combined by using + operator.

Example: Join two words with a space.

```
s1 = "Hello"  
s2 = "World"  
print(s1 + " " + s2)
```

× ▶ 📄

Output

Hello World

2. We can repeat a string multiple times using * operator.

Example: Repeat a greeting three times.

```
s = "Hello "  
print(s * 3)
```

× ▶ 📄

Output

Hello Hello Hello

Formatting Strings

1. **Using f-strings:** most preferred way to format strings is by using [f-strings](#).

Example: Embed variables directly using {} placeholders.

```
name = "Jake"  
age = 22  
print(f"Name: {name}, Age: {age}")
```

× ▶ 📄

Output

Name: Jake, Age: 22

2. **Using format():** Another way to format strings is by using [format\(\)](#) method.

Example: Use placeholders {} and pass values positionally.

```
s = "My name is {} and I am {} years old.".format( "Emily", 22)  
print(s)
```

× ▶ 📄

Output

My name is Emily and I am 22 years old.

String Membership Testing

`in` keyword checks if a particular substring is present in a string.

Example: Here, we are testing for the presence of substrings.

```
s = "GeeksforGeeks"
print("Geeks" in s)
print("GfG" in s)
```

× ▶ 📄

Output

```
True
False
```

Recommended Problems: [Repeat the Strings](#), [String Functions](#), [Convert String to LowerCase](#), [String Duplicates Removal](#), [Reverse String](#), [Check Palindrome](#)

Related Links:

- [String Comparison](#)
- [Convert integer to String](#)
- [Convert string to integer](#)
- [Convert a string to list](#)



Python String



String Operations in Python [Part -1]



String Operations in Python [Part -2]

Python String

Visit Course

Suggested Quiz

🔄 10 Questions

Which of the following is the correct way to create a string in Python?

- ☐ A str = "Hello"
- ☐ B str = 'Hello'

- ☐ C str = "Hello" or 'Hello'
- ☐ D All of the above

[Login to View Explanation](#)

1/10

< Previous Next >

Comment

S abhish...

304

Article Tags:

[Misc](#)[Python](#)[python-string](#)

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate
Tower, Sector- 136, Noida, Uttar
Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante
Apartment, Sector 137, Noida, Gautam
Buddh Nagar, Uttar Pradesh, 201305

Company

About Us
Legal
Privacy
Policy
Careers
Contact Us
Corporate
Solution
Campus
Training

Explore

POTD
Practice
Problems
Connect
Blogs
Upskill
Courses

Tutorials

Programming
Languages
DSA
Web
Technology
AI, ML &
Data Science
DevOps
CS Core
Subjects
GATE
School
Subjects
Software and
Tools

Courses

ML and Data
Science
DSA and
Placements
Web
Development
Data Science
Programming
Languages
DevOps &
Cloud
GATE
Trending
Technologies

Offline
Centers

Noida
Bengaluru
Hyderabad
Kolkata

Preparation
Corner

Interview
Corner
Aptitude
Puzzles
GfG 160
System Design



@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved